Praktikum 6 – Control Unit dan Register

NRP	: 1. 3223600038
	2. 3223600041
	3. 3223600048
	4. 3223600050
	5. 3223600058
Nama	: 1. Karolus Wira Wicaksana
	2. Akmal Rafiuddin
	3. Moch. Faris Afif
	4. Labib Al Afaf
	5. Ihsanta Zaki Sanjaya
Kelas	: 2 Teknik Komputer B
Matakuliah	: Praktikum Aplikasi Pengembangan Perangkat Lunak
Tanggal	: Rabu, 23 April 2025
Dosen	: Dr. Setiawardhana, ST. MT

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Program Control dan Register Output

6.1. Tujuan:

a. Mahasiswa dapat menjelaskan dan membuat program Program Control

b. Mahasiswa dapat menjelaskan dan membuat program Register Output

c. Mahasiswa dapat menjelaskan dan membuat program Peraga Biner

6.2. Dasar Teori:

A. Pengendali Pengurut (Control Unit)

Blok terbawah di bagian kiri berisi Pengendali Pengurut. Pengendali Pengurut atau Controller Sequencer yang disingkat **CON** merupakan unit pengendali keseluruhan. Sebelum komputer bekerja, sinyal-sinyal CLR masing-masing dikirimkan ke pencacah program dan register instruksi. Sebagai akibatnya, pencacah program direset ke 0000 dan bersamaan ini instruksi terakhir dalam register instruksi dihapus.

Sebuah sinyal detak CLK dikirimkan ke semua register bufer; sinyal ini mensinkronkan operasi komputer, yang menjamin bahwa setiap langkah operasi akan terjadi sebagaimana mestinya. Dengan kata lain, semua transfer dalam

regiter terjadi pada tepi positif dari sinyal detak CLK yang sama. Perhatikan bahwa sinyal CLK juga memasuki pencacah program.

Data 12-bit yang berasal dari Pengendali Pengurut membentuk suatu kata pengendalian komputer (seperti seorang pengawas yang memberitahu tentang apa yang harus dikerjakan). 12 kawat yang menyalurkan kata kendali itu disebut bus kendali (control bus). Kata kendali mempunyai format sbb:

$$\mathbf{CON} = C_p E_p \overline{L}_M \overline{CE} \quad \overline{L}_1 \overline{E}_1 \overline{L}_A E_A \quad S_u E_u \overline{L}_B \overline{L}_o$$

Kata ini menentukan bagaimana register-register harus bereaksi terhadap tepi positif sinyal detak berikutnya. Misalnya, E_p tinggi dan $\overline{L}_M$ rendah berarti bahwa isi pencacah program ditahan di dalam MAP pada tepi positif berikutnya. Sebagai contoh yang lain, CE dan $\overline{L}_A$ yang rendah berrati kata RAM yang telah ditunjuk alamatnya akan ditransfer ke akumulator pada tepi positif berikutnya dari lonceng. Kelak kita akan mempelajari diagram pewaktuan yang bersangkutan untuk melihat tepatnya kapan dan bagaimana transfer data ini dilakukan.

B. Register Output

Pada akhir operasi komputer, akumulator berisi jawaban dari persoalan yang diselesaikan. Pada saat ini, kita perlu memindahkan jawaban yang bersangkutan ke “dunia luar”. Untuk keperluan inilah register keluaran dipergunakan.

Apabila E_A tinggi
dan $\overline{L_0}$ rendah tepi positif sinyal detak berikutnya akan memasukkan kata dari akumulator ke dalam register keluaran.

C. Peraga Biner

Peraga biner adalah suatu barisan yang terdiri dari 8 buah LED. Oleh karena setiap LED dihubungkan dengan sebuah flip-flop dari register output, maka peraga biner akan menyajikan isi register outpu, kita dapat melihat jawaban itu dalam bentuk biner.

6.3. Tugas Pendahuluan:

a. Tuliskan tujuan praktikum
b. Apa yang anda ketahui tentang Register Output
c. Apa yang anda ketahui tentang Peraga Biner

6.4. Percobaan:

a. Membuat program Pengendali Pengurut (Control Unit)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity CU is
    Port (
        CLK       : in  STD_LOGIC;
        CLR       : in  STD_LOGIC;
        opcode    : in  STD_LOGIC_VECTOR (3 downto 0);
        ctrl_bus  : out STD_LOGIC_VECTOR (11 downto 0) :=
"001111100011";
        Xctrl_bus : out STD_LOGIC_VECTOR (11 downto 0) :=
"001111100011"
```

```
    );
end CU;

architecture Behavioral of CU is
    type state_type is (t1, t2, t3, t4, t5, t6);
    signal t_state: state_type;
begin

    process(CLK)
    begin
        if CLR = '1' then
            t_state <= t1;
        elsif rising_edge(CLK) then
            case t_state is
                when t1 => t_state <= t2;
                when t2 => t_state <= t3;
                when t3 => t_state <= t4;
                when t4 => t_state <= t5;
                when t5 => t_state <= t6;
                when t6 => t_state <= t1;
            end case;
        end if;
    end process;

    process(CLK)
    begin
        if falling_edge(CLK) then  -- Cp, Ep, nLm, nCe,
nLi, nEi, nLa, Ea, Su, Eu, nLb, nLo
            case opcode is
                when "0000" =>  -- LD A
                    case t_state is
                        when t1 => ctrl_bus <=
"010111100011"; Xctrl_bus <= "010111100011";
                        when t2 => ctrl_bus <=
"101111100011"; Xctrl_bus <= "101111100011";
                        when t3 => ctrl_bus <=
"001001100011"; Xctrl_bus <= "001001100011";
                        when t4 => ctrl_bus <=
"000110100011"; Xctrl_bus <= "000110100011";
                        when t5 => ctrl_bus <=
"001011000011"; Xctrl_bus <= "001011000011";
                        when t6 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";  -- NOP
                    end case;

                when "0001" =>  -- ADD
                    case t_state is
                        when t1 => ctrl_bus <=
"010111100011"; Xctrl_bus <= "010111100011";
```

```
                        when t2 => ctrl_bus <=
"101111100011"; Xctrl_bus <= "101111100011";
                        when t3 => ctrl_bus <=
"001001100011"; Xctrl_bus <= "001001100011";
                        when t4 => ctrl_bus <=
"000110100011"; Xctrl_bus <= "000110100011";
                        when t5 => ctrl_bus <=
"001011100001"; Xctrl_bus <= "001011100001";
                        when t6 => ctrl_bus <=
"001111000111"; Xctrl_bus <= "001111000111";
                    end case;

                when "0010" =>  -- SUBB
                    case t_state is
                        when t1 => ctrl_bus <=
"010111100011"; Xctrl_bus <= "010111100011";
                        when t2 => ctrl_bus <=
"101111100011"; Xctrl_bus <= "101111100011";
                        when t3 => ctrl_bus <=
"001001100011"; Xctrl_bus <= "001001100011";
                        when t4 => ctrl_bus <=
"000110100011"; Xctrl_bus <= "000110100011";
                        when t5 => ctrl_bus <=
"001011100001"; Xctrl_bus <= "001011100001";
                        when t6 => ctrl_bus <=
"001111001111"; Xctrl_bus <= "001111001111";
                    end case;

                when "1110" =>  -- OUT
                    case t_state is
                        when t1 => ctrl_bus <=
"010111100011"; Xctrl_bus <= "010111100011";
                        when t2 => ctrl_bus <=
"101111100011"; Xctrl_bus <= "101111100011";
                        when t3 => ctrl_bus <=
"001001100011"; Xctrl_bus <= "001001100011";
                        when t4 => ctrl_bus <=
"001111110010"; Xctrl_bus <= "001111110010";
                        when t5 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";
                        when t6 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";
                    end case;

                when "1111" =>  -- OUT
                    case t_state is
                        when t1 => ctrl_bus <=
```

```
"010111100011"; Xctrl_bus <= "010111100011";
                        when t2 => ctrl_bus <=
"101111100011"; Xctrl_bus <= "101111100011";
                        when t3 => ctrl_bus <=
"001001100011"; Xctrl_bus <= "001001100011";
                        when t4 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";
                        when t5 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";
                        when t6 => ctrl_bus <=
"001111100011"; Xctrl_bus <= "001111100011";
                    end case;

                when others =>
                    ctrl_bus <= "001111100011";
            end case;
        end if;
    end process;

end Behavioral;
```

CU (testbench)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity tb_CU is
end tb_CU;

architecture behavior of tb_CU is

    -- Komponen yang diuji
    component CU
        Port (
            CLK       : in  STD_LOGIC;
            CLR       : in  STD_LOGIC;
            opcode    : in  STD_LOGIC_VECTOR(3 downto
0);
            ctrl_bus  : out STD_LOGIC_VECTOR(11 downto
0);
            Xctrl_bus : out STD_LOGIC_VECTOR(11 downto
0)
        );
    end component;

    -- Sinyal internal testbench
```

```
    signal CLK       : STD_LOGIC := '0';
    signal CLR       : STD_LOGIC := '0';
    signal opcode    : STD_LOGIC_VECTOR(3 downto 0) :=
"0000";
    signal ctrl_bus  : STD_LOGIC_VECTOR(11 downto 0);
    signal Xctrl_bus : STD_LOGIC_VECTOR(11 downto 0);

    -- Clock period
    constant clk_period : time := 10 ns;

begin

    -- Instansiasi unit yang diuji (UUT)
    uut: CU
        port map (
            CLK       => CLK,
            CLR       => CLR,
            opcode    => opcode,
            ctrl_bus  => ctrl_bus,
            Xctrl_bus => Xctrl_bus
        );

    -- Clock generator
    clk_process :process
    begin
        while now < 500 ns loop
            CLK <= '0';
            wait for clk_period / 2;
            CLK <= '1';
            wait for clk_period / 2;
        end loop;
        wait;
    end process;

    -- Stimulus proses
    stim_proc: process
    begin
        -- Reset
        CLR <= '1';
        wait for clk_period;
        CLR <= '0';

        -- Coba instruksi LD A (0000)
        opcode <= "0000";
        wait for clk_period * 12;

        -- Coba instruksi ADD (0001)
        opcode <= "0001";
```

```
        wait for clk_period * 12;

        -- Coba instruksi SUB (0010)
        opcode <= "0010";
        wait for clk_period * 12;

        -- Coba instruksi OUT (1110)
        opcode <= "1110";
        wait for clk_period * 12;

        -- Coba instruksi OUT lain (1111)
        opcode <= "1111";
        wait for clk_period * 12;

        -- Selesai
        wait;
    end process;

end behavior;
```

b. Membuat program Register Output

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity outReg is
    Port (
        CLK   : in  std_logic;
        nLo   : in  std_logic;
        inReg : in  std_logic_vector(7 downto 0);
        LED   : out std_logic_vector(7 downto 0)
    );
end outReg;

architecture Behavioral of outReg is
begin
    process(CLK)
    begin
        if rising_edge(CLK) then
            if nLo = '0' then
                LED <= inReg;
            end if;
        end if;
    end process;
end Behavioral;
```

Register Output (testbench)

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY outReg_TB IS
END outReg_TB;

ARCHITECTURE behavior OF outReg_TB IS

    -- Component Declaration for Unit Under Test (UUT)
    COMPONENT outReg
        PORT(
            CLK   : IN std_logic;
            nLo   : IN std_logic;
            inReg : IN std_logic_vector(7 downto 0);
            LED   : OUT std_logic_vector(7 downto 0)
        );
    END COMPONENT;

    -- Signals
    signal CLK    : std_logic := '0';
    signal nLo    : std_logic := '0';
    signal inReg  : std_logic_vector(7 downto 0) :=
(others => '0');
    signal LED    : std_logic_vector(7 downto 0);

    -- Clock period definition
    constant CLK_period : time := 20 ns;

BEGIN

    -- Instantiate the Unit Under Test (UUT)
    uut: outReg PORT MAP (
        CLK   => CLK,
        nLo   => nLo,
        inReg => inReg,
        LED   => LED
    );

    -- Clock process
    CLK_process : process
    begin
        while true loop
            CLK <= '0';
```

```
            wait for CLK_period / 2;
            CLK <= '1';
            wait for CLK_period / 2;
        end loop;
    end process;

    -- Stimulus processes
    stim_proc : process
    begin
        wait for 10 ns;
        inReg(0) <= not inReg(0);
        wait;
    end process;

    stim_proc1 : process
    begin
        wait for 20 ns;
        inReg(1) <= not inReg(1);
        wait;
    end process;

    stim_proc2 : process
    begin
        wait for 30 ns;
        inReg(2) <= not inReg(2);
        wait;
    end process;

    stim_proc3 : process
    begin
        wait for 40 ns;
        inReg(3) <= not inReg(3);
        wait;
    end process;

    stim_proc4 : process
    begin
        wait for 50 ns;
        inReg(4) <= not inReg(4);
        wait;
    end process;

    stim_proc5 : process
    begin
        wait for 60 ns;
        inReg(5) <= not inReg(5);
        wait;
    end process;
```

```
    stim_proc6 : process
    begin
        wait for 70 ns;
        inReg(6) <= not inReg(6);
        wait;
    end process;

    stim_proc7 : process
    begin
        wait for 80 ns;
        inReg(7) <= not inReg(7);
        wait;
    end process;

    stim_proc8 : process
    begin
        wait for 90 ns;
        nLo <= not nLo;
        wait;
    end process;

END behavior;
```

c. Membuat tampilan Peraga Biner

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if
instantiating
-- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity peraga_biner is
    Port (
        SW  : in  STD_LOGIC_VECTOR(7 downto 0);  --
Input dari switch/DIP
        LED : out STD_LOGIC_VECTOR(7 downto 0)   --
Output ke LED
    );
end peraga_biner;
```

```
architecture Behavioral of peraga_biner is

begin

    -- Sinyal input langsung ditampilkan ke LED
    LED <= SW;

end Behavioral;
```

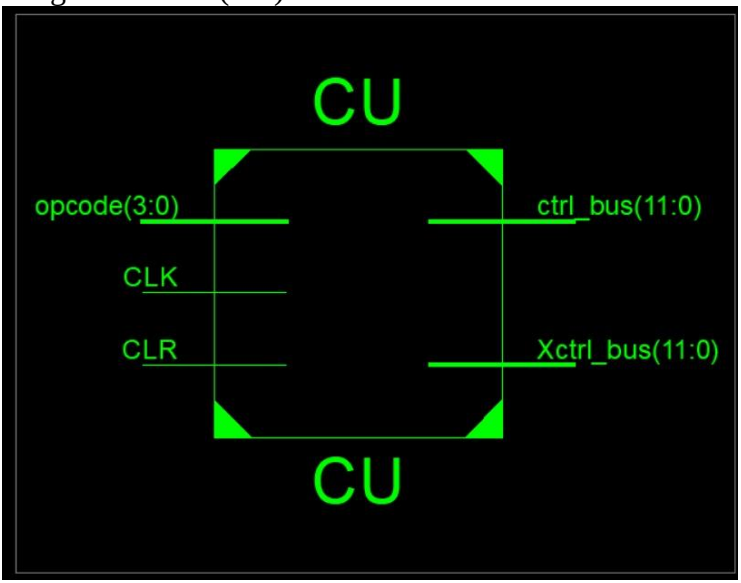
Peraga Biner (testbenchh)

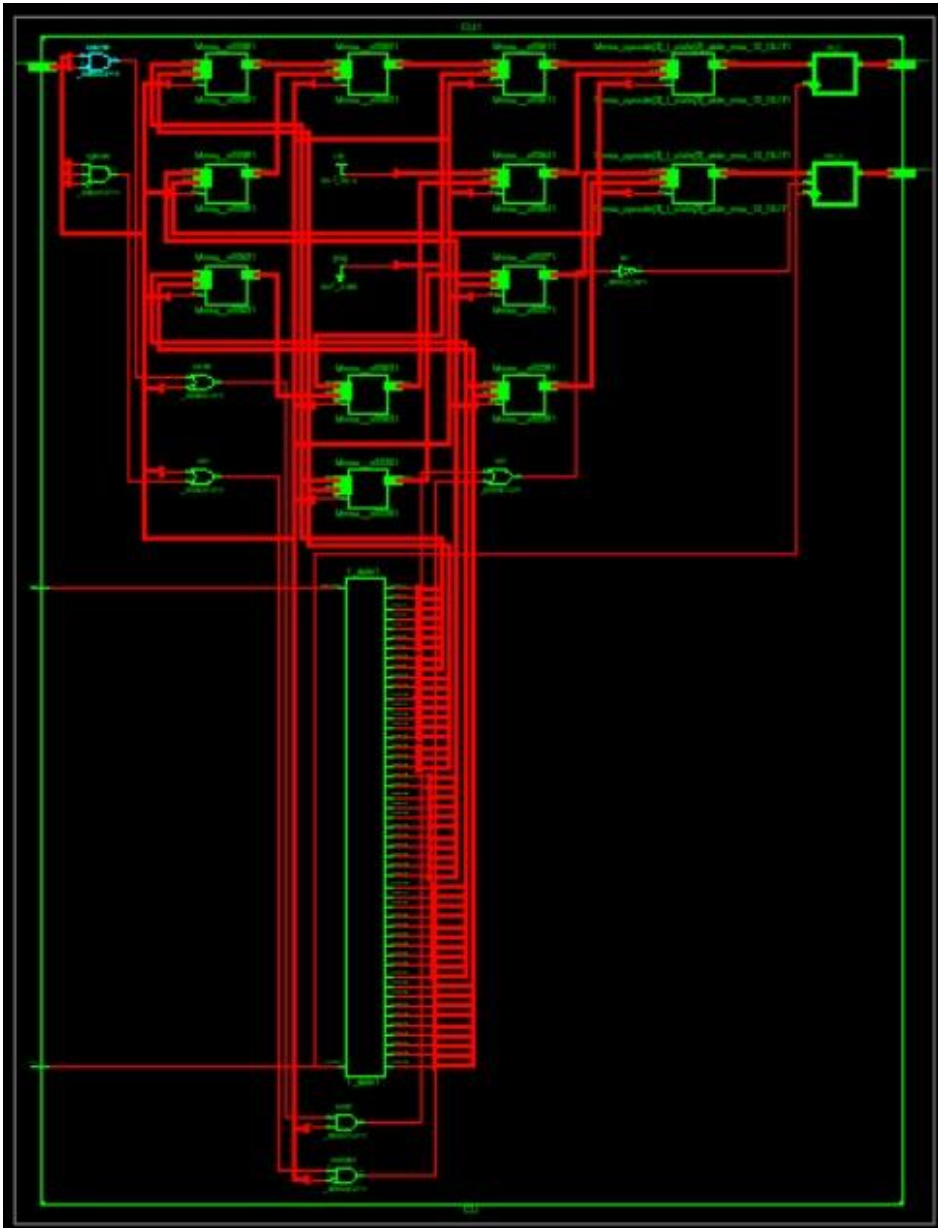
```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
--USE ieee.numeric_std.ALL;

ENTITY peraga_biner_tb IS
END peraga_biner_tb;

ARCHITECTURE behavior OF peraga_biner_tb IS



    COMPONENT peraga_biner
    PORT(
         SW : IN  std_logic_vector(7 downto 0);
         LED : OUT  std_logic_vector(7 downto 0)
        );
    END COMPONENT;


   --Inputs
   signal SW : std_logic_vector(7 downto 0) := (others
=> '0');

     --Outputs
   signal LED : std_logic_vector(7 downto 0);
   -- No clocks detected in port list. Replace <clock>
below with
BEGIN

     -- Instantiate the Unit Under Test (UUT)
   uut: peraga_biner PORT MAP (
          SW => SW,
          LED => LED
```

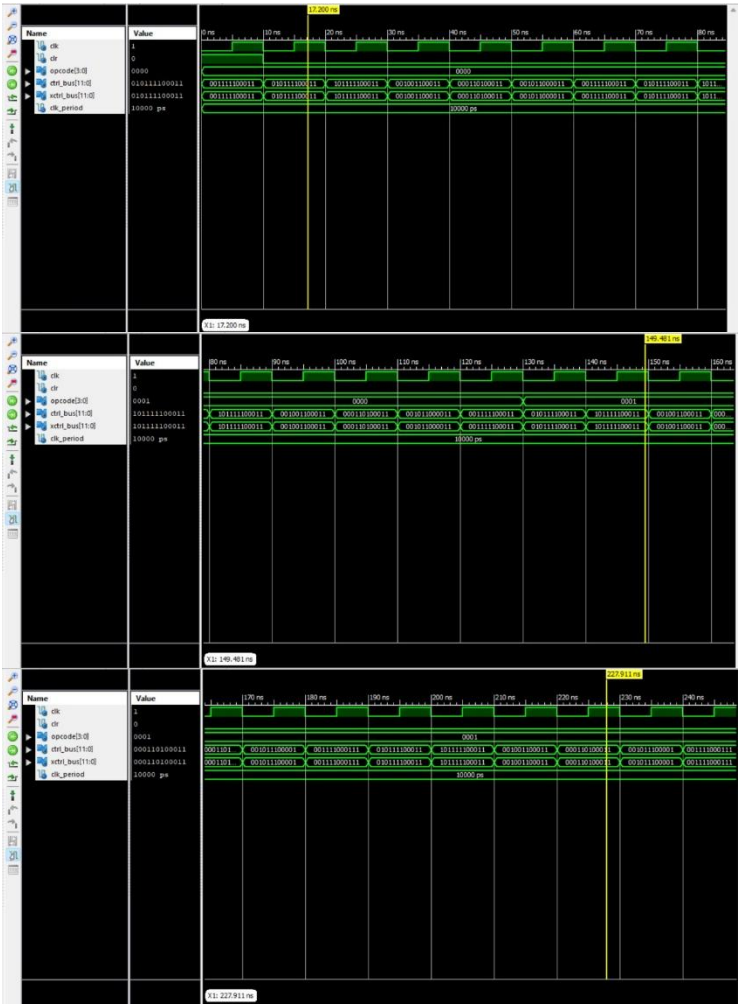
```
        );


    process
    begin
        -- Coba beberapa pola biner
        SW <= "00000000"; wait for 10 ns;
        SW <= "11111111"; wait for 10 ns;
        SW <= "10101010"; wait for 10 ns;
        SW <= "01010101"; wait for 10 ns;
        SW <= "11001100"; wait for 10 ns;
        SW <= "00110011"; wait for 10 ns;
        wait;
    end process;
END;
```

6.5. Hasil:

Pengendali Urut (CU)

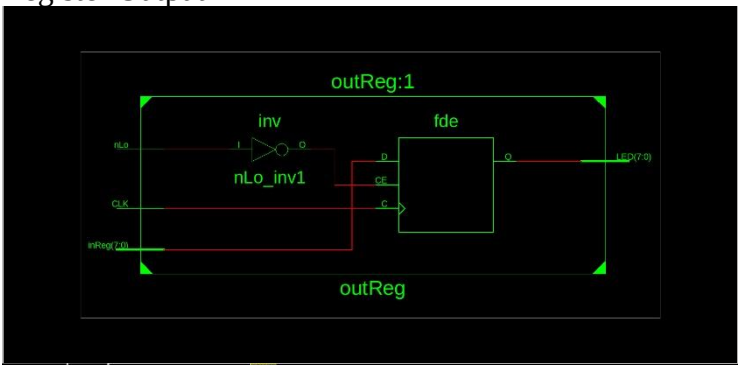


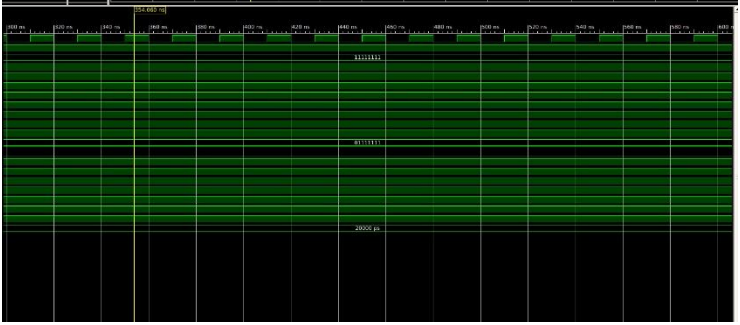


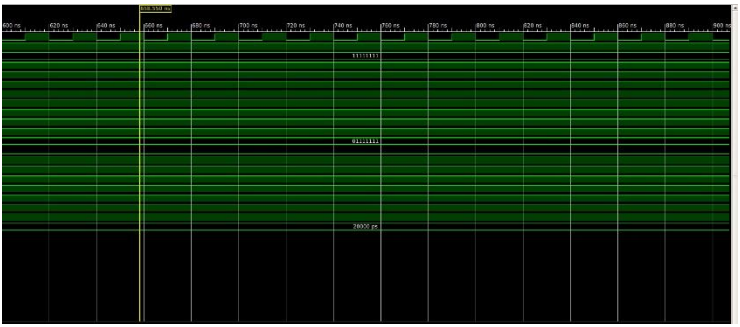

Name
Value
clk
clr
opcode[3:0]
ctrl_bus[11:0]
xctrl_bus[11:0]
clk_period
10000 ps
X1: 17.200 ns
X1: 149.481 ns
X1: 227.911 ns

Name
Value
clk
clr
opcode[3:0]
ctrl_bus[11:0]
xctrl_bus[11:0]
clk_period
10000 ps
X1: 255.503 ns
X1: 388.540 ns
X1: 468.087 ns

Register Output







Peraga Biner

6.6. Analisa:

Berdasarkan hasil simulasi testbench, unit kendali (Control Unit) telah berjalan sesuai rancangan state machine—sinyal t_state bergeser berurutan dari t1 hingga t6 dan kembali ke t1 tanpa anomali. Untuk setiap opcode (“0000” untuk LD A, “0001” untuk ADD, “0010” untuk SUBB, “1110” dan “1111” untuk OUT), nilai pada ctrl_bus dan Xctrl_bus pada tiap state selaras dengan kata kendali yang ditentukan, sehingga instruksi berhasil di-decode dan dieksekusi dengan benar. Tidak terlihat glitch atau tunda yang signifikan; seluruh alur sinyal kendali terkoordinasi oleh tepi jatuh (falling_edge(CLK)) serta sinyal reset (CLR) sesuai spesifikasi.

Register Output (outReg) juga berfungsi baik: pada setiap rising edge CLK ketika nLo aktif logika

rendah, data dari inReg langsung ter-transfer ke LED tanpa kehilangan bit. Pengujian pola biner pada inReg menunjukkan bahwa perubahan bit selalu tercermin tepat pada LED pada siklus berikutnya, membuktikan bahwa mekanisme latch bekerja secara sinkron. Modul peraga biner sederhana pun diuji dengan berbagai pola (misalnya 00000000, 11111111, 10101010), dan LED menyala persis mengikuti input SW, sehingga modul ini siap digunakan sebagai indikator visual dalam sistem.

6.7. Kesimpulan:

Praktikum ini berhasil mencapai tujuannya: mahasiswa mampu merancang dan mengimplementasikan Control Unit, Register Output, dan Peraga Biner sesuai teori bus kendali dan state machine sinyal sinkron. Ketiga modul telah tervalidasi secara fungsional dan siap digabungkan sebagai sub-sistem CPU sederhana, di mana Control Unit menghasilkan kata kendali, Register Output menyalurkan hasil akumulator ke dunia luar, dan Peraga Biner menampilkan hasil tersebut dalam bentuk LED biner.